

Database Query Guide (KCCF Clinic System)

This guide contains SQL queries that are:

- used by the current system,
- useful for learning database operations, and
- helpful for thesis/project defense discussion.

Database name:

```
USE kccf_clinic_db;
```

1. Core Tables Used by the System

- `users`
- `students`
- `medicines`
- `visits`
- `health_records`

2. Authentication Queries

2.1 Login user lookup (used in backend)

```
SELECT user_id, name, role, password, status
FROM users
WHERE username = ?;
```

Defense note:

- Uses parameterized query (?) to avoid SQL injection.
- Password is verified in PHP using `password_verify()`, not plain SQL comparison.

3. Dashboard and Summary Queries

3.1 Total students

```
SELECT COUNT(*) AS total_students
FROM students;
```

3.2 Today's visit count

```
SELECT COUNT(*) AS todays_visits
FROM visits
WHERE date_logged = CURRENT_DATE;
```

3.3 Critical stock count

```
SELECT COUNT(*) AS critical_medicines
FROM medicines
WHERE quantity <= 10;
```

3.4 Recent clinic activity

```
SELECT
    v.time_in,
    v.complaint,
    v.date_logged,
    s.first_name,
    s.last_name,
    hr.treatment
FROM visits v
JOIN students s ON v.student_id = s.student_id
LEFT JOIN health_records hr ON v.visit_id = hr.visit_id
ORDER BY v.date_logged DESC, v.time_in DESC
LIMIT 5;
```

4. Student Management Queries

4.1 Register new student

```
INSERT INTO students (
    first_name,
    last_name,
    gender,
    birthdate,
    course,
    year_level,
    guardian_name,
    relationship,
    guardian_contact
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?);
```

4.2 Update student profile

```
UPDATE students
SET
    first_name = ?,
    last_name = ?,
    gender = ?,
    birthdate = ?,
    course = ?,
    year_level = ?,
    guardian_name = ?,
    relationship = ?,
    guardian_contact = ?
WHERE student_id = ?;
```

4.3 Search students with filters and pagination

```
SELECT s.*
FROM students s
WHERE
    (s.first_name LIKE :search OR s.last_name LIKE :search OR s.student_id LIKE :search)
    AND (:course = '' OR s.course = :course)
    AND (:gender = '' OR s.gender = :gender)
ORDER BY s.student_id DESC
LIMIT :limit OFFSET :offset;
```

5. Visit and Triage Queries

5.1 Log new active visit

```
INSERT INTO visits (
    student_id,
    complaint,
    temperature,
    height,
    weight,
    nurse_notes,
    time_in,
    date_logged,
    status
) VALUES (?, ?, ?, ?, ?, ?, ?, CURRENT_DATE, 'Active');
```

5.2 Active queue listing

```
SELECT
    v.*, s.first_name, s.last_name, s.student_id AS real_student_id, s.course,
```

```
s.year_level
FROM visits v
JOIN students s ON v.student_id = s.student_id
WHERE v.status = 'Active'
ORDER BY v.date_logged ASC, v.time_in ASC
LIMIT :limit OFFSET :offset;
```

5.3 Visit log (completed only)

```
SELECT
    v.visit_id,
    v.complaint,
    v.temperature,
    v.height,
    v.weight,
    v.nurse_notes,
    v.time_in,
    v.date_logged,
    v.status,
    s.student_id AS real_student_id,
    s.first_name,
    s.last_name,
    s.course,
    s.year_level
FROM visits v
JOIN students s ON v.student_id = s.student_id
WHERE v.status = 'Completed'
ORDER BY v.date_logged DESC, v.time_in DESC
LIMIT :limit OFFSET :offset;
```

6. Finalize Consultation (Transaction Example)

This is one of the best queries to explain during defense because it shows ACID transaction behavior.

6.1 Begin transaction

```
START TRANSACTION;
```

6.2 Insert final health record

```
INSERT INTO health_records (visit_id, student_id, diagnosis, treatment)
VALUES (?, ?, ?, ?);
```

6.3 Mark visit as completed

```
UPDATE visits
SET status = 'Completed'
WHERE visit_id = ?;
```

6.4 Lock medicine row before stock deduction

```
SELECT quantity
FROM medicines
WHERE med_id = ?
FOR UPDATE;
```

6.5 Deduct stock

```
UPDATE medicines
SET quantity = quantity - ?
WHERE med_id = ?;
```

6.6 Commit or rollback

```
COMMIT;
-- or
ROLLBACK;
```

Defense note:

- **FOR UPDATE** prevents race conditions when two users dispense the same medicine at the same time.
- **ROLLBACK** protects data integrity if any step fails.

7. Inventory Queries

7.1 Add medicine

```
INSERT INTO medicines (name, category, quantity, status, expiration)
VALUES (?, ?, ?, ?, ?);
```

7.2 Restock medicine

```
UPDATE medicines
SET quantity = quantity + ?,
```

```
status = ?  
WHERE med_id = ?;
```

7.3 Update medicine details

```
UPDATE medicines  
SET  
    name = ?,  
    category = ?,  
    quantity = ?,  
    status = ?,  
    expiration = ?  
WHERE med_id = ?;
```

7.4 Low-stock monitoring

```
SELECT med_id, name, category, quantity, status, expiration  
FROM medicines  
WHERE quantity <= 10  
ORDER BY quantity ASC;
```

8. Reporting Queries (Defense-Ready)

8.1 Top complaints

```
SELECT complaint, COUNT(*) AS total  
FROM visits  
WHERE date_logged BETWEEN ? AND ?  
GROUP BY complaint  
ORDER BY total DESC  
LIMIT 5;
```

8.2 Visits by course

```
SELECT s.course, COUNT(*) AS total  
FROM visits v  
JOIN students s ON v.student_id = s.student_id  
WHERE v.date_logged BETWEEN ? AND ?  
GROUP BY s.course  
ORDER BY total DESC;
```

8.3 Gender breakdown

```
SELECT s.gender, COUNT(*) AS total
FROM visits v
JOIN students s ON v.student_id = s.student_id
WHERE v.date_logged BETWEEN ? AND ?
GROUP BY s.gender;
```

8.4 Peak clinic hours

```
SELECT
    SUM(CASE WHEN time_in < '12:00:00' THEN 1 ELSE 0 END) AS morning_cases,
    SUM(CASE WHEN time_in >= '12:00:00' THEN 1 ELSE 0 END) AS afternoon_cases
FROM visits
WHERE date_logged BETWEEN ? AND ?;
```

8.5 Fever surveillance

```
SELECT COUNT(*) AS fever_cases
FROM visits
WHERE temperature >= 37.8
    AND date_logged BETWEEN ? AND ?;
```

9. Useful Admin/Nurse Utility Queries

9.1 Distinct course list for filters

```
SELECT DISTINCT course
FROM students
WHERE course IS NOT NULL AND course != ''
ORDER BY course ASC;
```

9.2 Recent complaint suggestions (autocomplete)

```
SELECT complaint
FROM visits
WHERE complaint IS NOT NULL AND complaint != ''
GROUP BY complaint
ORDER BY MAX(visit_id) DESC
LIMIT 50;
```

9.3 Recent diagnosis suggestions (autocomplete)

```
SELECT diagnosis
FROM health_records
WHERE diagnosis IS NOT NULL AND diagnosis != ''
GROUP BY diagnosis
ORDER BY MAX(record_id) DESC
LIMIT 50;
```

10. Example Queries With Actual Values

Use these directly in MySQL for demo/practice.

10.1 Login lookup using real username

```
SELECT user_id, name, role, password, status
FROM users
WHERE username = 'admin';
```

10.2 Add a sample student record

```
INSERT INTO students (
    first_name,
    last_name,
    gender,
    birthdate,
    course,
    year_level,
    guardian_name,
    relationship,
    guardian_contact
) VALUES (
    'Paolo',
    'Delos Santos',
    'Male',
    '2007-06-15',
    'BSIT',
    2,
    'Maria Delos Santos',
    'Mother',
    '09170000001'
);
```

10.3 Update an existing student

```
UPDATE students
SET guardian_contact = '09175554444';
```



```
year_level = 3
WHERE student_id = 1;
```

10.4 Search active queue by complaint keyword

```
SELECT v.visit_id, s.first_name, s.last_name, v.complaint, v.time_in
FROM visits v
JOIN students s ON v.student_id = s.student_id
WHERE v.status = 'Active'
      AND v.complaint LIKE '%Allergic%'
ORDER BY v.time_in ASC;
```

10.5 Insert a new active visit

```
INSERT INTO visits (
    student_id,
    complaint,
    temperature,
    height,
    weight,
    nurse_notes,
    time_in,
    date_logged,
    status
) VALUES (
    1,
    'Headache',
    37.4,
    140.0,
    38.0,
    'Student reports headache after exam.',
    '08:45:00',
    CURRENT_DATE,
    'Active'
);
```

10.6 Finalize consultation with transaction (example)

```
START TRANSACTION;

INSERT INTO health_records (visit_id, student_id, diagnosis, treatment)
VALUES (61, 1, 'Tension Headache', 'Paracetamol 500mg given, advised hydration and rest.');
```

```
UPDATE visits
SET status = 'Completed'
```

```
WHERE visit_id = 61;

SELECT quantity
FROM medicines
WHERE med_id = 1
FOR UPDATE;

UPDATE medicines
SET quantity = quantity - 1
WHERE med_id = 1;

COMMIT;
```

10.7 Add a medicine item

```
INSERT INTO medicines (name, category, quantity, status, expiration)
VALUES ('Ibuprofen 200mg', 'NSAID / Pain Reliever', 40, 'In Stock', '2027-12-31');
```

10.8 Restock a medicine and update status

```
UPDATE medicines
SET quantity = quantity + 25,
    status = 'In Stock'
WHERE med_id = 2;
```

10.9 Report query using fixed date range

```
SELECT complaint, COUNT(*) AS total
FROM visits
WHERE date_logged BETWEEN '2026-01-01' AND '2026-05-05'
GROUP BY complaint
ORDER BY total DESC
LIMIT 5;
```

10.10 Fever monitoring for a month

```
SELECT COUNT(*) AS fever_cases
FROM visits
WHERE temperature >= 37.8
    AND date_logged BETWEEN '2026-04-01' AND '2026-04-30';
```

11. Defense Talking Points

You can use these points in your database defense:

1. The system uses relational design with foreign keys:
 - `visits.student_id -> students.student_id`
 - `health_records.visit_id -> visits.visit_id`
 - `health_records.student_id -> students.student_id`
2. Data integrity is enforced by constraints:
 - Primary keys for identity.
 - `ENUM` for controlled values (`role`, `status`, `gender`).
 - `NOT NULL` for required fields.
3. Security is handled using prepared statements:
 - Most user input goes through bound parameters (`?`, `:search`, etc.).
4. Concurrency is managed during medicine dispensing:
 - Uses transaction + `SELECT ... FOR UPDATE` + `COMMIT/ROLLBACK`.
5. Reporting is query-driven and reusable:
 - Date-range summaries, top complaints, demographics, and peak-hours analysis.

12. Optional Indexes (for performance demonstration)

These are optional but good for defense if asked about optimization:

```
CREATE INDEX idx_visits_status_date ON visits(status, date_logged);
CREATE INDEX idx_visits_student ON visits(student_id);
CREATE INDEX idx_health_records_student_date ON health_records(student_id, date);
CREATE INDEX idx_medicines_quantity ON medicines(quantity);
CREATE INDEX idx_students_course_year ON students(course, year_level);
```

Note:

- Add indexes only after testing because every index also adds write overhead.

13. Quick Test Queries for Panel Demo

```
-- Show active queue
SELECT visit_id, student_id, complaint, time_in, date_logged
FROM visits
WHERE status = 'Active'
ORDER BY date_logged, time_in;

-- Show latest completed health records
SELECT record_id, student_id, diagnosis, date
FROM health_records
```

```
ORDER BY date DESC
LIMIT 10;

-- Show critical medicines
SELECT med_id, name, quantity, status
FROM medicines
WHERE quantity <= 5
ORDER BY quantity ASC;
```

Prepared for: KCCF Clinic Management System database learning and defense reference.